Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2025

SIMULACIÓN DE PROPAGACIÓN DE ENFERMEDADES MEDIANTE TDA Y POO

202408977 – Jonathan Eduardo Fuentes García

Resumen

Este ensayo explica cómo se desarrolló un programa para simular la propagación de enfermedades en células, usando el lenguaje C# y programación orientada a objetos. La idea nace de un problema real: científicos necesitan analizar cómo se comportan las células contagiadas para predecir si una enfermedad será leve, grave o mortal. El programa usa listas enlazadas creadas desde cero (sin usar las de C#) para guardar la información de los pacientes y la evolución de las rejillas de células. También genera archivos XML para guardar los resultados y usa Graphviz para crear imágenes que muestran cómo cambia la rejilla período por período. Al final, el programa logra detectar patrones de repetición y clasificar correctamente la enfermedad según las reglas dadas.

Palabras clave

Simulation, OOP, linked lists, XML, Graphviz

Abstract

This paper explains how a program was developed to simulate disease spread in cells using C# language and object-oriented programming. The idea comes from a real problem: scientists need to analyze how infected cells behave to predict if a disease will be mild, severe or fatal. The program uses linked lists created from scratch (without using C# built-in lists) to store patient information and the evolution of cell grids. It also generates XML files to save results and uses Graphviz to create images showing how the grid changes period by period. Finally, the program manages to detect repetition patterns and correctly classify the disease according to the given rules.

El abstract y las keywords deben abarcar solamente esta columna.

Keywords

Simulation, OOP, linked lists, XML, Graphviz

.

Introducción

En este proyecto nos pidieron hacer un programa que simule cómo se propagan las enfermedades en células. La cosa es así: los científicos tienen rejillas cuadradas con células sanas (0) y contagiadas (1), y aplicando reglas como "una célula contagiada sigue contagiada si tiene 2 o 3 vecinos contagiados" o "una célula sana se contagia si tiene exactamente 3 vecinos contagiados", pueden predecir si la enfermedad será grave o mortal.

El chiste del programa es que pueda cargar estos datos desde un XML, hacer la simulación período por período, y detectar si los patrones se repiten. Si el patrón inicial se repite cada 1 período, el paciente muere. Si se repite cada más de 1 período, es grave. Y si no se repite nunca, es leve.

Desarrollo del tema

a. ¿Cómo guardamos los datos?

Lo primero era decidir cómo guardar toda la información. El proyecto exigía usar listas enlazadas creadas por nosotros mismos, nada de List<T> de C#. Así que tocó hacer nuestras propias clases:

- **NodoCelda**: guarda la posición (fila, columna) de una célula contagiada y un puntero al siguiente nodo.
- **ListaCeldas**: es una lista enlazada que va guardando todas las celdas contagiadas de una rejilla.
- **NodoEstado**: guarda el estado de una rejilla en un período específico (como un string de 0s y 1s).
- **ListaEstado**: va guardando todos los estados por los que va pasando la simulación.
- **NodoPaciente** y **ListaPacientes**: para manejar varios pacientes.

La verdad es que al principio me costó trabajo entender por qué había que hacer listas enlazadas si C# ya tiene estructuras más fáciles. Pero después me di cuenta de que era para aprender cómo funcionan por dentro y para practicar manejo de memoria y punteros.

b. La simulación paso a paso

La simulación fue lo más interesante. Por cada período, el programa:

1. Toma la rejilla actual
2. Cuenta los vecinos de cada celda
3. Aplica las reglas para determinar qué celdas siguen contagiadas
4. Genera la nueva rejilla

Lo complicado fue asegurarse de que los vecinos se contaran bien, especialmente en los bordes de la rejilla. Si no tenías cuidado, el programa intentaba acceder a posiciones que no existían y todo explotaba.

c. La detección de patrones

Aquí fue donde más me confundí al principio. La idea es que el programa guarda el estado de cada período como un string (ej: "010101..."). Cuando genera un nuevo período, revisa si ese estado ya apareció antes.

Si sí, significa que encontramos un patrón que se repite.

El cálculo de N y N1 fue lo más delicado:

- **N** es el período donde apareció el estado por primera vez
- **N1** es cada cuántos períodos se repite (período actual - N)

Y según las reglas:

- Si N =1 (el patrón inicial se repite en período 1) → **MORTAL**
- Si N1 =1 (cualquier patrón se repite cada 1 período) → **MORTAL**
- Si N1> 1 → **GRAVE**
- Si no hay repetición en los períodos máximos → **LEVE**

d. Los archivos XML

El programa tiene que leer y escribir XML. Para leer, usamos un CargadorXML que extrae los datos de cada paciente y las celdas contagiadas. Un detalle importante: en el XML las coordenadas empiezan en 1, pero en el programa manejamos desde 0. Al principio no lo consideré y todo salía corrido.

```
// Así corregimos las coordenadas
int fila = int.Parse(celda.Attributes["f"].Value) - 1;
int columna = int.Parse(celda.Attributes["c"].Value) - 1;
```

e. Las imágenes con Graphviz

Esta parte estuvo buena. Graphviz permite generar imágenes a partir de archivos .dot. Lo que hicimos fue que, por cada período de simulación, el programa crea un archivo .dot que representa la rejilla como una tabla. Las celdas contagiadas se ven en rojo y las sanas en blanco.

f. Problemas que encontramos en el camino

Como en todo proyecto, nos topamos con varios problemas:

1. **Coordenadas fuera de rango**: Por no restar 1 a las coordenadas del XML
2. **La simulación se detenía muy pronto**: Porque hacíamos return al encontrar el patrón
3. **No se guardaban todos los estados**: Relacionado con el punto anterior
4. **N1 mal calculado**: Por confundir la diferencia con el período actual

Cada problema nos enseñó algo nuevo. Por ejemplo, el de las coordenadas me hizo entender que siempre hay que revisar cómo vienen los datos de entrada.

g. Diagrama de clases

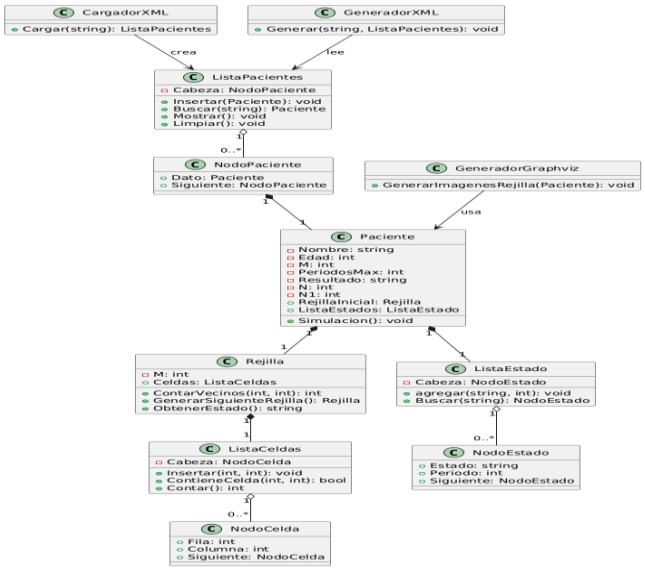

Figura 1. Diagrama de clases

h. Diagrama de actividades

-simulación de pasos

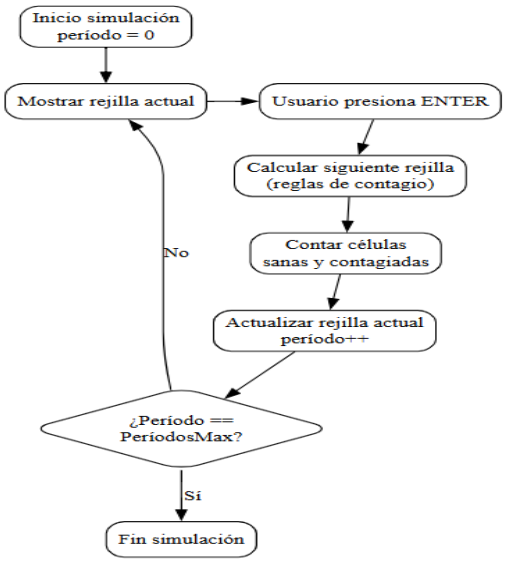

Figura 2. Diagrama simulación paso a paso
Fuente: elaborado en: Graphviz online

-Detección de Patrones (N y N1)

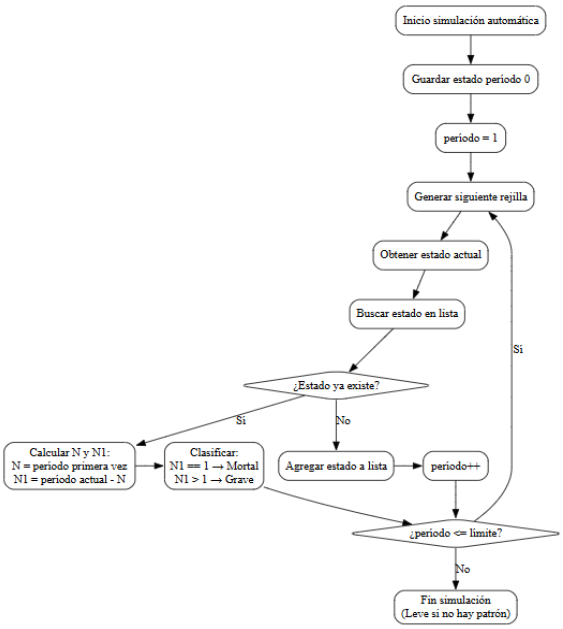

Figura 3. Diagrama detección de patrones
Fuente: elaborado en: Graphviz online

-Carga de XML

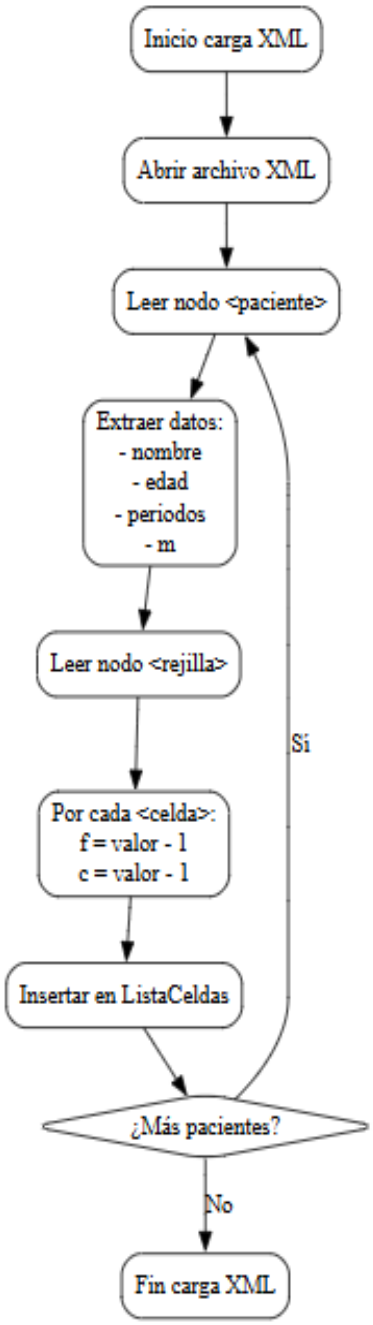

Figura 4. Diagrama Carga de XML
Fuente: elaborado en: Graphviz online

-Generación de Graphviz

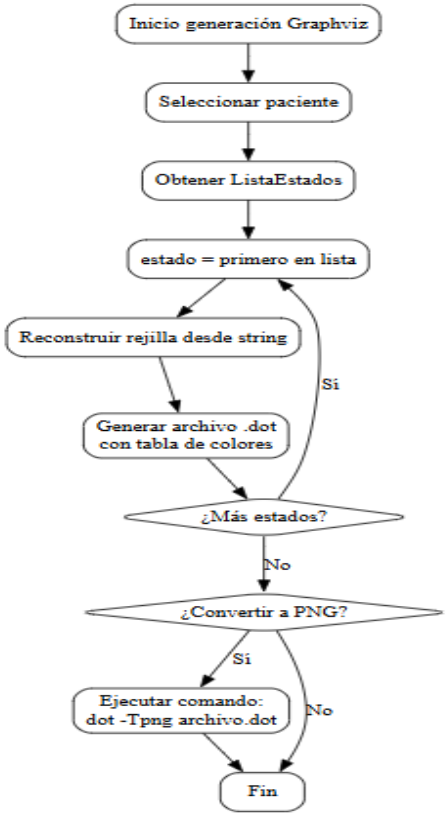

Figura 4. Diagrama generación de Graphviz
Fuente: elaborado en: Graphviz online

Conclusiones

Después de todo este proceso, puedo decir que:

1. **Las listas enlazadas sí tienen sentido**: Aunque al principio parecía complicarse la vida, entender cómo funcionan por dentro ayuda a comprender mejor la memoria y la eficiencia de los programas.
2. **La POO facilita las cosas**: Separar el programa en clases (Paciente, Rejilla, ListaCeldas, etc.) hizo que todo fuera más ordenado y fácil de modificar cuando encontrábamos errores.
3. **Los patrones se pueden detectar**: El método de guardar estados como strings y compararlos resultó efectivo para encontrar ciclos en la simulación.
4. **Graphviz es útil para visualizar**: Aunque al principio no entendía bien para qué servía, ver las imágenes de cómo evolucionan las células período por período ayuda mucho a entender el comportamiento de la enfermedad.
5. **Los detalles pequeños importan**: Un error en las coordenadas, un return mal puesto, y todo el programa falla. Aprendí a revisar línea por línea cuando algo no funciona.

Referencias bibliográficas

Facultad de Ingeniería, Universidad San Carlos de Guatemala. (2026). *Introducción a la Programación y Computación 2 - Proyecto 1*. Guatemala.

Graphviz. (2024). *Graph Visualization Software*. https://graphviz.org/

Microsoft. (2024). *Documentación de C#*. https://docs.microsoft.com/es-es/dotnet/csharp/